

به نام خدا

[نام دانشگاه]

[نام دانشکده]

پایان نامه‌ی تحصیلی مقطع کارشناسی در رشته‌ی مهندسی نرم افزار

طراحی و پیاده سازی سامانه‌ی مدیریت کتابخانه‌ی مبتنی بر وب به همراه
چارچوب آزمون خودکار چندسطحی

Design and Implementation of a Web-Based Library Management System with a Multi-Level Automated Testing Framework

[نام استاد راهنما]

استاد راهنما:

[نام و نام خانوادگی نگارنده]

نگارنده:

شهریور ۱۴۰۵

چکیده

کیفیت نرم‌افزار امروزه به یکی از دغدغه‌های اصلی تیم‌های توسعه تبدیل شده و آزمون خودکار به‌عنوان راهکاری بنیادین برای تضمین رفتار صحیح سامانه‌ها در برابر تغییرات پیوسته‌ی کد به شمار می‌رود. در این پایان‌نامه، طراحی و پیاده‌سازی یک سامانه‌ی مدیریت کتابخانه‌ی مبتنی بر وب به‌همراه یک چارچوب آزمون خودکار چندسطحی ارائه می‌شود. کاربرد موردی با نام Aldenwood Library در دو بخش توسعه یافته است: پیشنهادی کاربری بر پایه‌ی Next.js و React و سرویس‌دهنده‌ی بر پایه‌ی FastAPI، SQLAlchemy و پایگاه داده‌ی SQLite. تمرکز اصلی پروژه بر خودِ کاربرد نیست، بلکه بر طراحی سازوکار آزمونی است که در سه سطح آزمون واحد (۷۱ مورد)، آزمون یکپارچگی (۳۵ مورد) و آزمون پایانه‌به‌پایه (۱۳ مورد) رفتار سامانه را پوشش می‌دهد.

چارچوب پیشنهادی بر پایه‌ی pytest ساخته شده است، برای آزمون‌های رابط کاربری از الگوی Page Object و مرورگر Selenium بهره می‌گیرد، با ابزارهای Hypothesis، factory-boy و freezegun غنی‌سازی شده و پوشش خط حدود ۹۱ درصدی را در برابر آستانه‌ی اجباری ۸۵ درصد به دست می‌آورد. اجرای آزمون‌ها در خط لوله‌ی ادغام مداوم گیت‌هاب اکشنز نیز خودکارسازی شده است؛ به‌گونه‌ای که آزمون‌های پشتیبان در هر push و آزمون‌های رابط کاربری به‌صورت شبانه اجرا می‌شوند. نتایج نشان می‌دهد گذر از بازبینی دستی به آزمون خودکار، زمان هر سیکل بازبینی را از حدود ۱۱ دقیقه به حدود ۳۰ ثانیه کاهش می‌دهد، نقطه‌ی سربه‌سر هزینه پس از حدود سه سیکل رگرسیون حاصل می‌شود و امکان کشف زودهنگام رگرسیون‌ها پیش از ادغام تغییرات فراهم می‌گردد.

واژگان کلیدی: آزمون خودکار نرم‌افزار، هرم آزمون، آزمون واحد، آزمون یکپارچگی، آزمون پایانه‌به‌پایه، سلنیوم، ادغام مداوم

Abstract

Software quality has become one of the main concerns of development teams, and automated testing is regarded as a fundamental means of ensuring correct system behaviour in the presence of continuous code change. This thesis presents the design and implementation of a web-based library management system together with a multi-level automated testing framework. The case-study application, named Aldenwood Library, was developed in two parts: a front end built on Next.js and React, and a back end built on FastAPI, SQLAlchemy and an SQLite database. The main focus of the project is not the application itself but the design of a testing mechanism that covers system behaviour at three levels: unit tests (۷۱ cases), integration tests (۳۰ cases) and end-to-end tests (۱۳ cases).

The proposed framework is built on pytest, employs the Page Object pattern with Selenium for UI testing, is enriched with factory-boy, Hypothesis and freezegun, and achieves approximately ۹۱% line coverage against an enforced ۸۵% gate. Test execution is further automated in a GitHub Actions continuous-integration pipeline: back-end tests run on every push, while UI tests run nightly. The results show that replacing manual verification with the automated suite reduces the time of each regression pass from roughly ۱۱ minutes to roughly ۳۰ seconds, breaks even on the initial investment after about three regression cycles, and enables early detection of regressions before changes are merged.

Keywords: Automated Software Testing, Test Pyramid, Unit Testing, Integration Testing, End-to-End Testing, Selenium, Continuous Integration

فهرست مطالب

توجه: این فهرست با کدهای میدانی ورد تولید شده است؛ پس از ویرایش سند، برای به‌روزرسانی شماره صفحه‌ها روی فهرست راست کلیک کرده و گزینه‌ی

Update Field را برگزینید.

I	چکیده
II	Abstract
۱	فصل اول: کلیات پژوهش
۱	۱-۱ مقدمه
۱	۱-۲ بیان مسئله
۲	۱-۳ اهداف پروژه
۳	۱-۴ دامنه‌ی پروژه
۳	۱-۵ روش کار
۳	۱-۶ ساختار پایان‌نامه
۵	فصل دوم: مبانی نظری و مرور پیشینه
۵	۲-۱ آزمون نرم‌افزار: تعریف و اهمیت
۵	۲-۲ سطوح آزمون نرم‌افزار
۶	۲-۳ هرم آزمون
۷	۲-۴ توسعه‌ی آزمون‌محور و ادغام مداوم
۸	۲-۵ فناوری‌های سامانه‌های وب مدرن و آزمون‌پذیری
۹	۲-۶ جمع‌بندی فصل
۱۰	فصل سوم: طراحی و پیاده‌سازی سامانه‌ی مدیریت کتابخانه
۱۰	۳-۱ معماری کلی سامانه
۱۱	۳-۲ پشته‌ی فناوری
۱۲	۳-۳ طراحی پایگاه داده
۱۵	۳-۴ لایه‌ی کسب‌وکار و قواعد امانت‌دهی
۱۹	۳-۵ رابط برنامه‌نویسی کاربردی و کنترل دسترسی
۲۰	۳-۶ امنیت احراز هویت
۲۱	۳-۷ رابط کاربری

.....۲۱	۳-۸ داده‌های اولیه
.....۲۲	۳-۹ جمع‌بندی فصل
.....۲۳	فصل چهارم: طراحی و پیاده‌سازی چارچوب آزمون خودکار
.....۲۳	۴-۱ استراتژی و ساختار چارچوب
.....۲۴	۴-۲ آزمون‌های واحد
.....۲۵	۴-۳ آزمون‌های یکپارچگی
.....۲۷	۴-۴ آزمون‌های پایانه‌به‌پایه
.....۲۸	۴-۵ گزارش‌گیری و پوشش کد
.....۲۹	۴-۶ مقایسه‌ی بازبینی دستی و آزمون خودکار
.....۳۰	۴-۷ خط لوله‌ی ادغام مداوم
.....۳۲	۴-۸ جمع‌بندی فصل
.....۳۳	فصل پنجم: نتایج، نتیجه‌گیری و پیشنهادها
.....۳۳	۵-۱ مرور دستاوردها
.....۳۴	۵-۲ پاسخ به اهداف پژوهش
.....۳۴	۵-۳ محدودیت‌ها و پیشنهادهاى توسعه
.....۳۵	۵-۴ نتیجه‌گیری
.....۳۶	مراجع
.....۳۷	پیوست الف: راهنمای نصب و اجرا
.....۳۸	پیوست ب: واژه‌نامه

فهرست جداول

۶	جدول ۱-۲: سطوح آزمون نرم‌افزار و ویژگی‌های آن‌ها در پروژه‌ی حاضر
۱۱	جدول ۱-۳: پشته‌ی فناوری سامانه به تفکیک لایه
۱۴	جدول ۲-۳: جدول‌های پایگاه داده و شرح آن‌ها
۱۶	جدول ۳-۳: پارامترهای انواع عضویت در داده‌های اولیه
۱۹	جدول ۴-۳: قواعد کلیدی کسب‌وکار سامانه و پارامترهای آن‌ها
۲۰	جدول ۵-۳: نقش‌های کاربری و سطح دسترسی در سامانه
۲۴	جدول ۱-۴: توزیع آزمون‌ها به تفکیک سطح و فایل
۲۹	جدول ۲-۴: مقایسه‌ی بازبینی دستی و مجموعه‌ی آزمون خودکار
۳۱	جدول ۳-۴: گام‌های خط لوله‌ی ادغام مداوم پروژه
۳۴	جدول ۱-۵: محدودیت‌های کنونی و پیشنهادهای توسعه‌ی آتی
۳۷	جدول الف-۱: گام‌های نصب و اجرای پروژه
۳۸	جدول ب-۱: واژه‌نامه‌ی اصطلاحات فنی پایان‌نامه

فهرست اشکال

- شکل ۱-۲: هرم آزمون چارچوب خودکار پروژه (تعداد آزمون‌های هر سطح) ۷
- شکل ۱-۳: معماری لایه‌ای سامانه: از پیشانه‌ی کاربری تا پایگاه داده ۱۰
- شکل ۲-۳: نمودار موجودیت-رابطه‌ی پایگاه داده (۱۱ جدول شامل جدول واسط book_authors) ۱۳
- شکل ۳-۳: ماشین حالت وضعیت نسخه‌ی کتاب (وضعیت‌های DAMAGED و IN_REPAIR در این نسخه تولید نمی‌شوند) ۱۶
- شکل ۴-۳: فلوچارت تصمیم‌گیری سرویس امانت‌دهی در ثبت امانت ۱۸
- شکل ۱-۴: توزیع ۱۱۹ آزمون سازمان‌یافته‌ی چارچوب بر اساس سطح ۲۳
- شکل ۲-۴: پوشش خط به‌دست‌آمده در برابر آستانه‌ی اجباری اجرا ۲۹
- شکل ۳-۴: مقایسه‌ی زمان هر گذر کامل بازبینی دستی و مجموعه‌ی آزمون خودکار ۳۰

فصل اول: کلیات پژوهش

۱-۱ مقدمه

گسترش روزافزون کاربردهای مبتنی بر وب و افزایش سرعت تغییر نیازهای کاربران، کیفیت نرم‌افزار را به عاملی تعیین‌کننده در موفقیت پروژه‌های توسعه تبدیل کرده است. مطالعات مهندسی نرم‌افزار نشان می‌دهد که کشف عیوب در مراحل نخست تولید، هزینه‌ای بسیار کمتر از کشف آن‌ها پس از استقرار دارد و آزمون منظم و تکرارپذیر، سازوکار اصلی این کشف زودهنگام است [۱] و [۲]. در همین راستا، آزمون خودکار که در قالب چارچوب‌های استاندارد اجرا می‌شود، امروزه جزء جدایی‌ناپذیر فرایند توسعه‌ی نرم‌افزار نوین به شمار می‌رود.

پروژه‌ی موضوع این پایان‌نامه دو مؤلفه‌ی به‌هم‌پیوسته دارد. مؤلفه‌ی نخست، سامانه‌ی مدیریت کتابخانه‌ی مبتنی بر وب Aldenwood Library است که عملیات امانت، بازگشت، تمدید، رزرو و مدیریت منابع کتابخانه را فراهم می‌کند. مؤلفه‌ی دوم و محور اصلی پروژه، چارچوب آزمون خودکار چندسطحی‌ای است که صحت رفتار این سامانه را در سطوح واحد، یکپارچگی و پایانه‌به‌پایه به‌صورت تکرارپذیر راستی‌آزمایی می‌کند. به بیان دیگر، تمرکز پروژه بیش از آنکه بر خودِ کاربرد کتابخانه‌ای باشد، بر طراحی یک راه‌حل آزمون خودکارِ نگه‌داشت‌پذیر است؛ زیرا ارزش بلندمدت هر سامانه‌ی نرم‌افزاری به سهولت تغییر امن آن وابسته است [۱۸].

۱-۲ بیان مسئله

در سامانه‌هایی مانند نرم‌افزارهای کتابخانه‌ای، منطق کسب‌وکار ظاهراً ساده اما در عمل درهم‌تنیده است: سقف امانت هر نوع عضویت، محاسبه‌ی جریمه‌ی دیرکرد، مهلت تمدید، صف رزرو و قواعد ویژه‌ی منابع مرجع، همگی بر یکدیگر اثر می‌گذارند. راستی‌آزمایی دستی چنین قواعدی پیش از هر تغییر، کاری

زمان‌بر، پرهزینه و مستعد خطای انسانی است و پس از چند سیکل توسعه عملاً کنار گذاشته می‌شود؛ نتیجه‌ی آن ورود بی‌سروصدا رگرسیون‌ها به محصول است.

مسئله‌ی اصلی این پژوهش را می‌توان چنین صورت‌بندی کرد: «چگونه می‌توان برای یک سامانه‌ی وب دارای منطق کسب‌وکار چندلایه، چارچوب آزمون خودکاری طراحی کرد که در سه سطح واحد، یکپارچگی و پایانه‌به‌پایه عمل کند، پوشش آزمون را در سطح قابل قبولی تضمین نماید، رفتارهای حساسی چون رقابت همزمانی و امنیت توکن‌های احراز هویت را پوشش دهد و در خط لوله‌ی ادغام مداوم اجرا شود؟» پاسخ به این پرسش، خروجی‌هایی چون معماری لایه‌ای تست‌پذیر برای سامانه، مجموعه‌ای سازمان‌یافته از ۱۱۹ آزمون خودکار، سازوکار گزارش‌گیری و پوشش کد و خط لوله‌ی CI را در پی دارد.

۳-۱ اهداف پروژه

اهداف تعیین‌شده برای این پروژه به شرح زیر است:

۱. طراحی و پیاده‌سازی سامانه‌ی مدیریت کتابخانه‌ی وب با معماری لایه‌ای و تست‌پذیر (پیشانه‌ی

Next.js و سرویس‌دهنده‌ی FastAPI)؛

۲. طراحی استراتژی آزمون بر مبنای هرم آزمون و پیاده‌سازی سه سطح آزمون واحد، یکپارچگی و پایانه‌به‌پایه؛

۳. پوشش قواعد کسب‌وکار حساس شامل سقف امانت، جریمه‌ی دیرکرد، تمدید، رزرو و رقابت همزمانی بر سر آخرین نسخه‌ی کتاب؛

۴. راستی‌آزمایی امنیت احراز هویت مبتنی بر توکن در برابر مجموعه‌ای از حملات متداول توکن؛

۵. خودکارسازی آزمون رابط کاربری با Selenium و الگوی Page Object در محیط مرورگر بی‌سر؛

۶. تضمین پوشش خط حداقل ۸۵ درصدی به‌صورت اجباری در ابزار آزمون و خط لوله؛

۷. اجرای خودکار آزمون‌ها در گیت‌هاب اکشنز و ارائه‌ی گزارش‌های HTML، JSON و پوشش کد به‌عنوان خروجی‌های قابل ردیابی.

۴-۱ دامنه‌ی پروژه

دامنه‌ی پروژه شامل توسعه‌ی کاربرد وب برای مرورگرهای رومیزی، پیاده‌سازی سرویس‌دهنده‌ی REST با احراز هویت مبتنی بر توکن، طراحی ۱۱ جدول پایگاه داده، تولید داده‌های اولیه‌ی واقع‌نما و طراحی و اجرای چارچوب آزمون سه‌سطحی است. پوشش آزمون، خودکارسازی CI و مستندسازی استراتژی آزمون نیز در قلب این دامنه قرار دارند. در مقابل، مواردی چون بهینه‌سازی برای تلفن همراه، آزمون بار و کارایی، ارزیابی دسترس‌پذیری و استقرار عملیاتی در محیط تولید، از دامنه‌ی این پروژه خارج‌اند و در فصل پنجم به‌عنوان مسیرهای توسعه‌ی آتی پیشنهاد شده‌اند.

۵-۱ روش کار

روند اجرای پروژه از الگوهای توسعه‌ی چابک الهام گرفته شده است [۹] و در شش گام پیش رفت: نخست، مطالعه‌ی مبانی نظری آزمون نرم‌افزار و بررسی فناوری‌های گزینه‌ی پشته‌ی توسعه؛ دوم، طراحی معماری لایه‌ای سامانه و مدل داده؛ سوم، پیاده‌سازی سرویس‌دهنده و پیشانه‌ی کاربری؛ چهارم، طراحی استراتژی آزمون و پیاده‌سازی آزمون‌ها به موازات توسعه‌ی امکان‌ها؛ پنجم، پایدارسازی آزمون‌های مرورگر در محیط بی‌سر و اجرای شبانه در CI؛ و ششم، ارزیابی نتایج شامل پوشش کد، مقایسه‌ی زمانی با بازیابی دستی و مستندسازی. در سراسر پروژه، هر قاعده‌ی کسب‌وکار تازه پیش از گسترش امکان‌های دیگر، با آزمون تثبیت شده است؛ رویکردی همسو با اصول توسعه‌ی آزمون‌محور [۷].

۶-۱ ساختار پایان‌نامه

ادامه‌ی این پایان‌نامه در پنج فصل سازمان یافته است. فصل دوم به مبانی نظری آزمون نرم‌افزار شامل سطوح آزمون، هرم آزمون و مفاهیم ادغام مداوم اختصاص دارد. فصل سوم طراحی و پیاده‌سازی سامانه‌ی

مدیریت کتابخانه را از منظر معماری، مدل داده، قواعد کسب‌وکار، امنیت و رابط کاربری شرح می‌دهد. فصل چهارم به‌طور مفصل چارچوب آزمون خودکار را در سه سطح و به‌همراه گزارش‌گیری، مقایسه با بازیابی دستی و خط لوله‌ی CI بررسی می‌کند. فصل پنجم دستاوردها، پاسخ به اهداف، محدودیت‌ها و پیشنهادهای توسعه را در بر می‌گیرد. در پایان نیز فهرست مراجع و دو پیوست شامل راهنمای اجرا و واژه‌نامه ارائه شده است.

فصل دوم: مبانی نظری و مرور پیشینه

۲-۱ آزمون نرم‌افزار: تعریف و اهمیت

آزمون نرم‌افزار فرایندی نظام‌مند برای اجرای یک مؤلفه یا سامانه با هدف یافتن عیب، سنجش کیفیت و ایجاد اطمینان از رفتار صحیح آن است [۳] و [۵]. استاندارد ISO/IEC/IEEE ۲۹۱۱۹ آزمون را بخشی از فرایند مهندسی نرم‌افزار می‌داند که ورودی‌ها، خروجی‌ها، محیط و معیارهای توقف مشخصی دارد [۵]. از منظر اقتصادی، ارزش آزمون در کاهش هزینه‌ی عیوب است: هرچه عیب دیرتر کشف شود، هزینه‌ی اصلاح آن به‌طور نمایی رشد می‌کند؛ چراکه عیب کشف‌شده پس از استقرار افزون بر اصلاح کد، اصلاح داده، اعتبار و پشتیبانی را نیز درگیر می‌کند [۲].

نقش کلیدی آزمون خودکار در تثبیت رفتار در برابر تغییر است: مجموعه آزمون خودکار مانند تور ایمنی عمل می‌کند و به توسعه‌دهنده اجازه می‌دهد ساختار کد را بدون ترس از خرابی‌های پنهان بهبود دهد [۱۸]. در این میان، آزمون رگرسیون — اجرای مجدد آزمون‌ها پس از هر تغییر برای اطمینان از دست‌نخورده‌ی رفتارهای پیشین — پرهزینه‌ترین نوع آزمون در حالت دستی و بیشترین سود را از خودکارسازی می‌برد [۳].

۲-۲ سطوح آزمون نرم‌افزار

سند سیلابس ISTQB آزمون‌ها را بر پایه‌ی دامنه‌ی بررسی به چهار سطح آزمون مؤلفه (واحد)، آزمون یکپارچگی، آزمون سامانه و آزمون پذیرش طبقه‌بندی می‌کند [۴]. در توسعه‌ی نرم‌افزار وب، سه سطح نخست بیشترین سهم خودکارسازی را دارند. جدول ۱-۲ این سطوح را به‌همراه ویژگی‌ها و نمونه‌ی متناظر آن‌ها در پروژه‌ی حاضر نشان می‌دهد.

جدول ۱-۲: سطوح آزمون نرم‌افزار و ویژگی‌های آن‌ها در پروژه‌ی حاضر

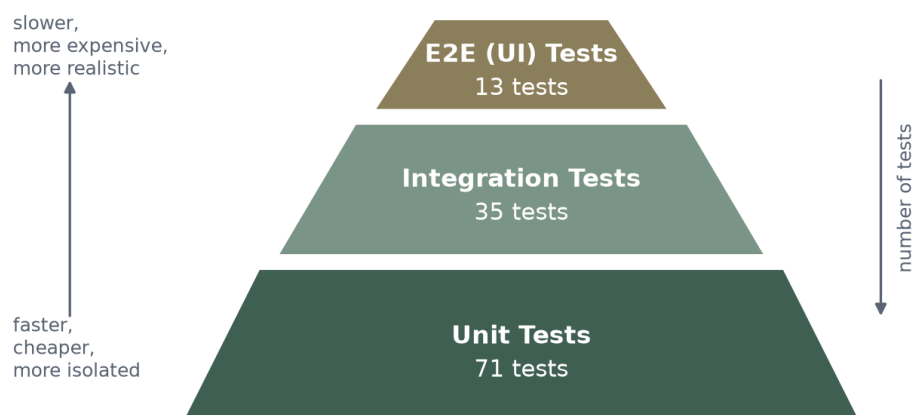
سطح آزمون	دامنه‌ی بررسی	وابستگی به محیط	سرعت اجرا	نمونه‌ی متناظر در این پروژه
آزمون واحد	توابع و کلاس‌های منطق کسب‌وکار، به‌صورت مجزا از وابستگی‌ها	ندارد؛ وابستگی‌ها با شیء ماک جایگزین می‌شوند	بسیار سریع	آزمون سرویس امانت‌دهی با اشیاء ماک
آزمون یکپارچگی	تعامل اجزا: مسیر کامل درخواست HTTP تا پایگاه داده	پایگاه داده‌ی آزمونی ایزوله در حافظه	سریع	آزمون چرخه‌ی امانت و بازگشت با TestClient
آزمون پایانه‌به‌پایه	رفتار کل سامانه از دید کاربر نهایی	مرورگر واقعی و سرورهای فعال	کند	سناریوهای Selenium با الگوی Page Object

منبع: بر پایه‌ی طبقه‌بندی [۴] ISTQB و مستندات استراتژی آزمون پروژه

۲-۳ هرم آزمون

هرم آزمون که نخستین‌بار توسط مایک کوهن طرح شد و بعدها مارتین فاولر آن را تثبیت کرد، راهنمای توزیع تعداد آزمون‌ها میان سطوح گوناگون است [۶]. مطابق شکل ۱-۲، قاعده‌ی هرم را آزمون‌های واحد فراوان، سریع و ارزان تشکیل می‌دهد؛ در میانه، آزمون‌های یکپارچگی با تعداد کمتر تعامل اجزای کلیدی را می‌آزمایند و در رأس، آزمون‌های پایانه‌به‌پایه‌ی کم‌شمار جریان‌های حیاتی کاربر را پوشش می‌دهند. منطق هرم، موازنه‌ی میان سرعت، هزینه و واقع‌نمایی است: هرچه به رأس نزدیک‌تر شویم، آزمون واقعی‌تر اما کندتر، پرهزینه‌تر و شکننده‌تر می‌شود.

Test Pyramid of the Automated Testing Framework



شکل ۲-۱: هرم آزمون چارچوب خودکار پروژه (تعداد آزمون‌های هر سطح)

الگوی نابهنجارِ مخروط وارونه — که در آن آزمون‌های رابط کاربری سنگین بر آزمون‌های واحد چیره‌اند — به تأخیر بازخورد، ناپایداری آزمون‌ها و دشواری عیب‌یابی می‌انجامد [۶]. در چارچوب حاضر، نسبت ۷۱ واحد، ۳۵ یکپارچگی و ۱۳ پایانه‌به‌پایه به‌آگاهانه بر مبنای همین اصل انتخاب شده است؛ این توزیع در فصل چهارم به تفصیل تحلیل می‌شود.

۲-۴ توسعه‌ی آزمون‌محور و ادغام مداوم

توسعه‌ی آزمون‌محور (TDD) چرخه‌ی سه‌گامی «نوشتن آزمون شکست‌خورده، نوشتن کمینه‌ی کدِ گذراننده‌ی آزمون، و بازآرایی» را بنیاد کار گذاشته می‌کند [۷]. فایده‌ی اصلی این چرخه، تضمین آزمون‌پذیری طراحی و شکل‌گیری مجموعه‌ای از آزمون‌های رگرسیون به‌موازات خود محصول است. در سطح فرایند، ادغام مداوم (CI) به معنای ادغام مکرر تغییرات در شاخه‌ی اصلی و اجرای خودکار ساخت و آزمون پس از هر ادغام است تا عیوب زودتر و با هزینه‌ی کمتر کشف شوند [۸].

در پروژه‌ی حاضر، هر دو مفهوم به کار گرفته شده است: قواعد کسب و کار در هر گام با آزمون تثبیت شده‌اند و خط لوله‌ی گیت‌هاب اکشنز در هر push آزمون‌های پشتیبان و در زمان‌بندی شبانه آزمون‌های رابط کاربری را اجرا می‌کند؛ ترکیبی که بازخورد سریع به توسعه‌دهنده و پایش دوره‌ای رفتار کل سامانه را هم‌زمان فراهم می‌آورد.

۵-۲ فناوری‌های سامانه‌های وب مدرن و آزمون‌پذیری

سامانه‌های وب مدرن معمولاً با معماری «پیشانه-سرویس‌دهنده» ساخته می‌شوند: پیشانه‌ی کاربرد تک‌صفحه‌ای^۱ در مرورگر اجرا می‌شود و با سرویس‌دهنده از طریق رابط برنامه‌نویسی کاربردی REST تبادل داده می‌کند. در سوی سرویس‌دهنده، نگاشت شیء-رابطه‌ای^۲ لایه‌ی دسترسی به داده را انتزاعی می‌کند. از منظر آزمون‌پذیری، این تفکیک ذاتی یک مزیت مهم دارد: منطق کسب و کار سرویس‌دهنده بدون مرورگر و بدون رابط کاربری — و در نتیجه با آزمون‌های واحد و یکپارچگی سریع — آزمون‌پذیر است و تنها جریان‌های تعامل کاربر نیازمند آزمون مرورگر واقعی‌اند.

افزون بر تفکیک لایه‌ها، الگوهای معماری نرم‌افزار مانند تزریق وابستگی و الگوی مخزن، آزمون‌پذیری را از «تصادف طراحی» به «ویژگی عمدی معماری» تبدیل می‌کنند [۱۷]. این اصل راهنمای طراحی سامانه‌ی فصل سوم است: سرویس‌های منطق کسب و کار هیچ وابستگی مستقیمی به چارچوب وب ندارند و همه‌ی وابستگی‌ها از بیرون به آن‌ها تزریق می‌شود.

^۱ برنامه‌ی تک‌صفحه‌ای (Single Page Application – SPA) به کاربردی گفته می‌شود که به جای بارگذاری مجدد کامل صفحه در هر درخواست، کل تعامل کاربر را در یک صفحه‌ی واحد و با تغییر پویای محتوا مدیریت می‌کند؛ برای این کار از درخواست‌های پس‌زمینه‌ای به سرویس‌دهنده‌ی REST استفاده می‌شود.

^۲ نگاشت شیء-رابطه‌ای (Object-Relational Mapping – ORM) فناوری‌ای است که جدول‌های رابطه‌ای را به کلاس‌های شیء‌گرا نگاشت می‌کند تا توسعه‌دهنده به جای نوشتن پرس‌وجوی SQL خام، با اشیاء برنامه کار کند؛ SQLAlchemy یکی از شناخته‌شده‌ترین ORM‌های زبان پایتون است.

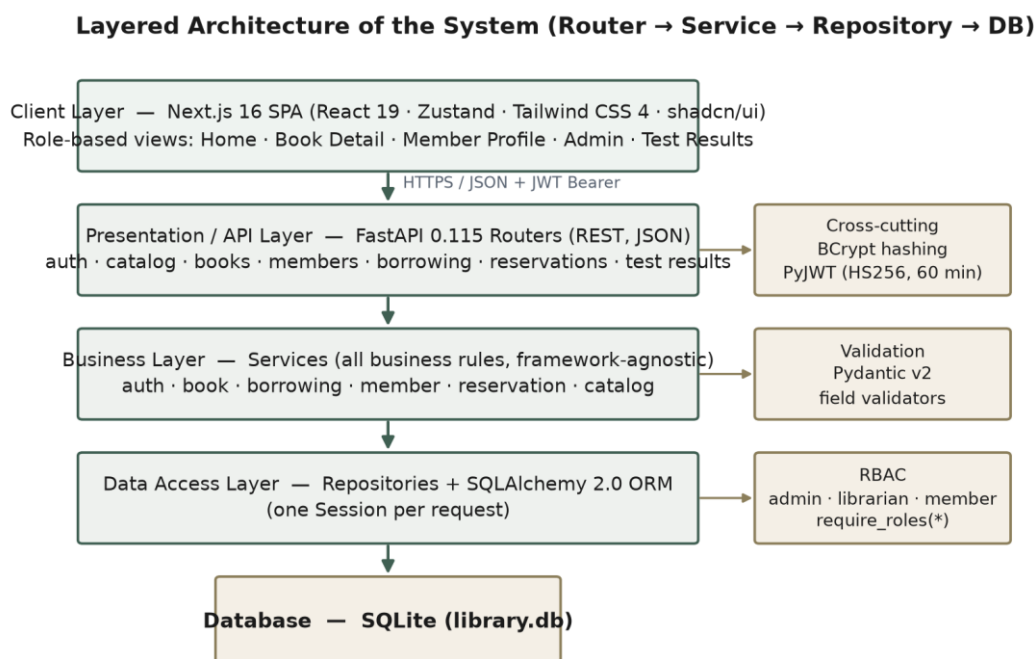
۶-۲ جمع‌بندی فصل

در این فصل، تعریف و اهمیت اقتصادی آزمون نرم‌افزار، طبقه‌بندی سطوح آزمون، اصل هرم آزمون و مفاهیم توسعه‌ی آزمون‌محور و ادغام مداوم مرور شد. بر این پایه، چارچوب نظری لازم برای داوری درباره‌ی تصمیم‌های طراحی در دو فصل بعد فراهم است: معماری لایه‌ای سامانه برای آزمون‌پذیری، توزیع آزمون‌ها بر مبنای هرم، و خودکارسازی اجرا در خط لوله‌ی CI.

فصل سوم: طراحی و پیاده‌سازی سامانه‌ی مدیریت کتابخانه

۳-۱ معماری کلی سامانه

سامانه بر پایه‌ی معماری لایه‌ای طراحی شده است که در شکل ۱-۳ نمایش داده شده است. درخواست‌های HTTP در لایه‌ی مسیرب‌های FastAPI دریافت می‌شوند و پس از اعتبارسنجی ورودی، به سرویس‌های منطق کسب‌وکار سپرده می‌شوند؛ سرویس‌ها عملیات داده‌ای را از طریق مخزن‌های داده^۳ و با میانجی‌گری ORM اجرا می‌کنند و پاسخ را به شکل مدل‌های خروجی بازمی‌گردانند. زنجیره‌ی «مسیرب ← سرویس ← مخزن ← نشست پایگاه داده» ستون فقرات سرویس‌دهنده است و هر لایه تنها با لایه‌ی مجاور خود گفت‌وگو می‌کند.



شکل ۱-۳: معماری لایه‌ای سامانه: از پیشنهادی کاربری تا پایگاه داده

^۳ الگوی مخزن (Repository Pattern) میان منطق کسب‌وکار و جزئیات دسترسی به داده یک لایه‌ی انتزاعی ایجاد می‌کند؛ به این ترتیب سرویس‌ها بدون آگاهی از جزئیات پایگاه داده و به‌صورت مجزا آزمون می‌شوند.

دو تصمیم طراحی در این معماری اثر مستقیمی بر آزمون‌پذیری دارد. نخست آنکه سرویس‌های منطق کسب‌وکار هیچ وابستگی‌ای به FastAPI، درخواست HTTP یا نشست پایگاه داده ندارند؛ همه‌ی وابستگی‌ها (مخزن‌ها، ساعت سیستم و شناسه‌ها) از بیرون تزریق می‌شوند. دوم آنکه استثناهای دامنه‌ای (یافت‌نشده، تکراری، ناقض قاعده‌ی کسب‌وکار، احراز‌نشده و فاقد مجوز) در لایه‌ی سرویس پرتاب می‌شوند و تنها در لایه‌ی اصلی برنامه به پاسخ‌های HTTP (به ترتیب ۴۰۴، ۴۰۰، ۴۰۹/۴۰۰، ۴۰۱ و ۴۰۳) ترجمه می‌شوند. به این ترتیب کل منطق کسب‌وکار بدون چارچوب وب و تنها با اشیاء ماک آزمون‌پذیر است — همان ویژگی‌ای که آزمون‌های واحد فصل چهارم بر آن استوارند.

۲-۳ پشته‌ی فناوری

جدول ۱-۳ فناوری‌های به‌کاررفته را به تفکیک لایه و همراه با نقش هر یک نشان می‌دهد. معیار انتخاب، ترکیبی از مدرن‌بودن، مستندات قوی، اکوسیستم آزمون غنی و سبکی راه‌اندازی بوده است؛ برای نمونه FastAPI به‌لطف اعتبارسنجی خودکار مبتنی بر Pydantic و پشتیبانی بومی مستندات تعاملی OpenAPI، گزینه‌ای مناسب برای سرویس‌دهنده‌ی آزمون‌پذیر است [۱۰]^۴ و Next.js نیز به‌عنوان چارچوب پیشنهادی React، ساخت پیشانه‌ی مبتنی بر مؤلفه را با تجربه‌ی توسعه‌ی یکپارچه فراهم می‌کند [۱۱].

جدول ۱-۳: پشته‌ی فناوری سامانه به تفکیک لایه

لایه	فناوری	نسخه	نقش در سامانه
پیشانه‌ی کاربری	Next.js + React	۱۶.۳.۳ / ۱۹.۲.۸	چارچوب کاربرد تک‌صفحه‌ای و رندر مؤلفه‌ها
پیشانه‌ی کاربری	TypeScript	۵.۹.۳	نوع‌دهی ایستا و آینه‌سازی مدل‌های Pydantic
پیشانه‌ی کاربری	Tailwind CSS + shadcn/ui	۴.۳.۳	استایل‌دهی و مؤلفه‌های رابط کاربری

^۴ اعتبارسنجی سطح ورودی شامل قواعدی چون: شماره‌ی استاندارد فقط عددی با طول ۱۰ یا ۱۳ پس از حذف خط تیره، قیمت مثبت، حداقل یک نویسنده برای هر کتاب و حداقل طول گذرواژه‌ی شش نویسه است.

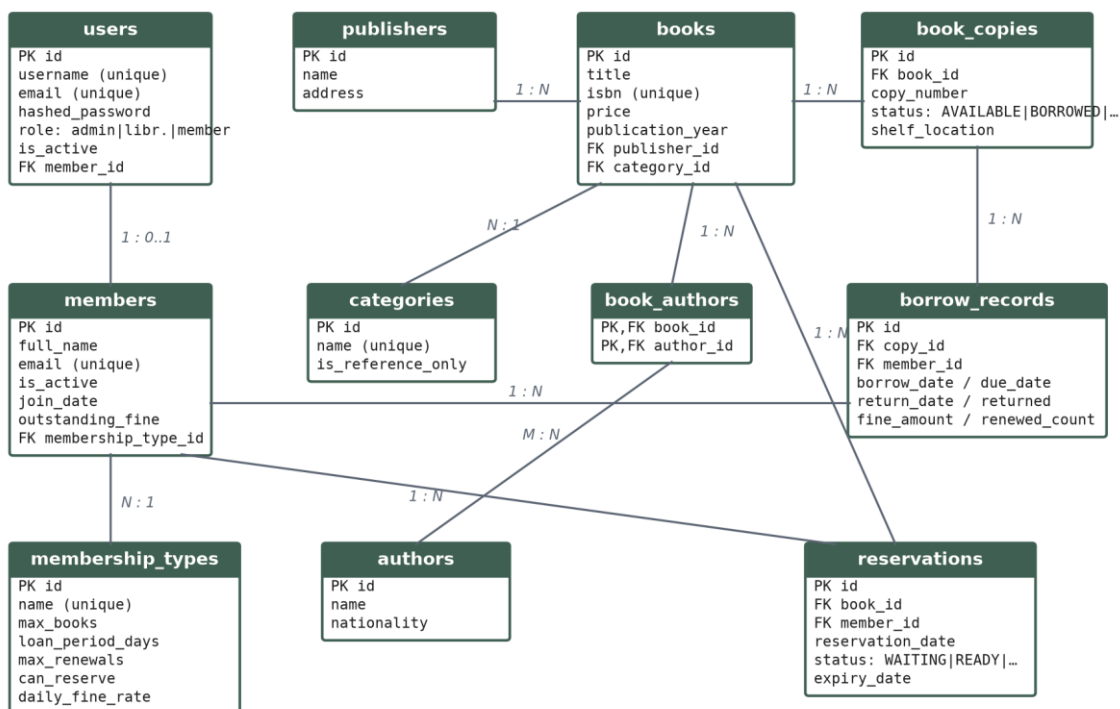
لایه	فناوری	نسخه	نقش در سامانه
پیشانه‌ی کاربری	Zustand	۵.۰.۱۵	مدیریت وضعیت سراسری (ورود، داده، ناوبری)
سرویس‌دهنده	Python	۳.۱۲	زبان پیاده‌سازی سرویس‌دهنده و چارچوب آزمون
سرویس‌دهنده	FastAPI + Uvicorn	۰.۱۱۵.۰ / ۰.۳۰.۶	چارچوب REST و اجراساز [۱۰] ASGI
سرویس‌دهنده	SQLAlchemy	۲.۰.۳۵	ORM و مدیریت نشست‌های پایگاه داده [۱۲]
سرویس‌دهنده	Pydantic	۲.۹.۲	اعتبارسنجی ورودی/خروجی و مدل‌های داده
داده	SQLite	—	پایگاه داده‌ی رابطه‌ای فایل‌محور (library.db)
امنیت	bcrypt / PyJWT	۴.۲.۰ / ۲.۹.۰	درهم‌سازی گذرواژه و امضای توکن [۱۵]
آزمون	pytest / Selenium	۸.۳.۳ / ۴.۲۵.۰	اجرای آزمون‌ها و خودکارسازی مرورگر [۱۳] و [۱۴]
آزمون	factory-boy / Hypothesis / freezegun	۳.۳.۱ / ۶.۱۶۷.۱ / ۱.۵.۵	داده‌ساز، آزمون ویژگی‌محور و قفل زمان

منبع: فایل package.json پروژه‌ی پیشانه و requirements.txt چارچوب آزمون

۳-۳ طراحی پایگاه داده

مدل داده‌ی سامانه شامل یازده جدول است که روابط میان آن‌ها در نمودار موجودیت-رابطه‌ی شکل ۳-۲ نمایش داده شده است. هسته‌ی مدل را سه موجودیت «کتاب»، «نسخه‌ی کتاب» و «عضو» تشکیل می‌دهند: موجودیت کتاب اطلاعات کتاب‌شناختی را نگه می‌دارد و هر نسخه‌ی فیزیکی آن به‌صورت رکوردی جداگانه با وضعیت مستقل (موجود، امانت‌داده‌شده، رزرو شده، گم‌شده و غیره) مدل شده است. این تفکیک کتاب از نسخه، پیش‌شرط مدل‌سازی صحیح امانت است؛ زیرا امانت به نسخه‌ی مشخصی از کتاب داده می‌شود، نه به عنوان کتاب.

Entity-Relationship Diagram of the Database (11 tables incl. junction table book_authors)



شکل ۳-۲: نمودار موجودیت-رابطه‌ی پایگاه داده (۱۱ جدول شامل جدول واسط book_authors)

جدول ۳-۲ شرح هر جدول و مهم‌ترین فیلدهای آن را جمع‌بندی می‌کند. سه نکته‌ی طراحی در این مدل درخواست تأکید دارد: نخست، رابطه‌ی چندبه‌چند میان کتاب و نویسنده با جدول واسط book_authors پیاده شده است تا هر کتاب چند نویسنده و هر نویسنده چند کتاب داشته باشد. دوم، نوع عضویت به‌صورت جدولی پارامتریک مدل شده است تا سقف امانت، مدت امانت، سقف تمدید، مجوز رزرو و نرخ جریمه بدون تغییر کد قابل تنظیم باشند. سوم، برای جلوگیری از امانت مضاعف یک نسخه، شاخص یکتای جزئی^۵ به‌نام uq_active_borrow_per_copy روی ستون copy_id در جدول امانت‌ها تعریف شده است که تنها ردیف‌های بازگشت‌نشده را محدود می‌کند؛ این شاخص در فصل چهارم، سنگ‌بنای آزمون رقابت همزمانی خواهد بود. گزیده‌ای از این تعریف در برنامه ۳-۱ آمده است.

^۵ شاخص یکتای جزئی (Partial Unique Index) شاخصی است که تنها بر ردیف‌های واجد شرط اعمال می‌شود؛ در این پروژه شرط returned = ۰ تضمین می‌کند که هر نسخه در هر لحظه حداکثر یک امانت فعال داشته باشد.

برنامه ۳-۱: تعریف شاخص یکتای جزئی در مدل امانت (SQLAlchemy)

```
class BorrowRecord(Base):
    __tablename__ = "borrow_records"

    id = Column(Integer, primary_key=True)
    copy_id = Column(ForeignKey("book_copies.id"), nullable=False)
    member_id = Column(ForeignKey("members.id"), nullable=False)
    borrow_date = Column(DateTime, nullable=False)
    due_date = Column(DateTime, nullable=False)
    returned = Column(Boolean, default=False)

    # at most one ACTIVE loan per copy, enforced by the database itself
    __table_args__ = (
        Index("uq_active_borrow_per_copy", "copy_id", unique=True,
            sqlite_where=text("returned = 0"),
            postgresql_where=text("returned = 0")),
    )
```

منبع: کد منبع پروژه (app/models/borrow.py)

جدول ۳-۲: جدول‌های پایگاه داده و شرح آن‌ها

جدول	شرح	کلیدها و فیلدهای مهم
users	حساب‌های کاربری ورود به سامانه	username, role (admin/librarian/member), email یکتا, FK member_id اختیاری
members	اطلاعات اعضای کتابخانه	full_name, outstanding_fine, is_active, FK membership_type_id
membership_types	انواع عضویت و پارامترهای آن‌ها	max_books, loan_period_days, max_renewals, can_reserve, daily_fine_rate
books	اطلاعات کتاب‌شناختی کتاب‌ها	category_id و price, FK publisher_id, title, isbn یکتا

جدول	شرح	کلیدها و فیلدهای مهم
book_copies	نسخه‌های	copy_number، status شامل
	فیزیکی هر	AVAILABLE/BORROWED/RESERVED/LOST/DAMAGED/IN_R
	کتاب	EPAIR
book_authors	جدول	
	واسط	کلید مرکب (book_id, author_id) برای رابطه‌ی چندبه‌چند
	کتاب-نویسنده	
authors	نویسندگان	name، nationality
	کتاب‌ها	
publishers	ناشران	name، address
categories	موضوع‌بندی	name یکتا، is_reference_only برای منابع مرجع
	کتاب‌ها	
borrow_records	سابقه‌ی	FK copy_id و member_id، due_date، return_date
	امانت و	renewed_count، fine_amount، شاخص جزئی
	بازگشت	uq_active_borrow_per_copy
reservations	رزرو	
	کتاب‌های	status شامل WAITING/READY/FULFILLED/CANCELLED
	بدون نسخه‌ی آزاد	expiry_date

منبع: کلاس‌های ORM در مسیر app/models پروژه‌ی سرویس‌دهنده

۳-۴ لایه‌ی کسب‌وکار و قواعد امانت‌دهی

قلب سامانه، سرویس امانت‌دهی است که قواعد سخت‌گیرانه‌ای را بر اجرای عملیات حاکم می‌کند.

جدول ۳-۳ پارامترهای سه نوع عضویت تعریف‌شده در داده‌های اولیه را نشان می‌دهد؛ همه‌ی قواعد کمی

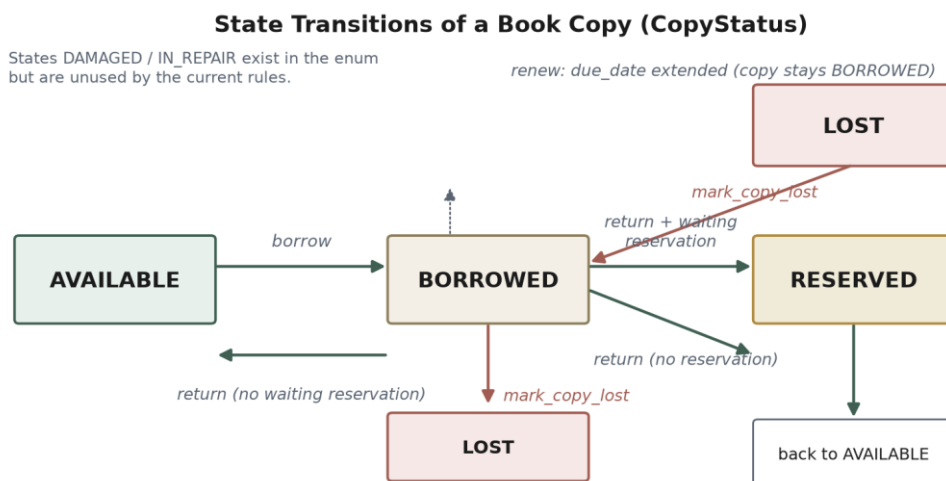
سامانه از همین پارامترها تغذیه می‌شوند و به همین دلیل تغییر سیاست کتابخانه نیازمند تغییر کد نیست.

جدول ۳-۳: پارامترهای انواع عضویت در داده‌های اولیه

نوع عضویت	حداکثر کتاب	مدت امانت (روز)	سقف تمدید	جریمه‌ی روزانه (دلار)	مجوز رزرو
Student	۳	۱۴	۱	۰.۵۰	بله
Regular	۵	۲۱	۲	۱.۰۰	بله
Premium	۱۰	۳۰	۳	۱.۵۰	بله

منبع: اسکریپت داده‌های اولیه seed_db.py

وضعیت هر نسخه‌ی کتاب در طول عمر خود در ماشین حالت شکل ۳-۳ جابه‌جا می‌شود. نکته‌ی ظریف این ماشین، گذار «بازگشت همراه با صف رزرو» است: اگر هنگام بازگشت نسخه، رزروی در انتظار برای آن کتاب وجود داشته باشد، نسخه به‌جای «موجود» به وضعیت «رزرو شده» می‌رود و قدیمی‌ترین رزرو در انتظار به وضعیت «آماده» با مهلت دو روزه ارتقا می‌یابد؛ سازوکاری که انصاف صف رزرو (اولین ورود، نخستین خدمت) را تضمین می‌کند.



شکل ۳-۳: ماشین حالت وضعیت نسخه‌ی کتاب (وضعیت‌های DAMAGED و IN_REPAIR در این نسخه تولید نمی‌شوند)

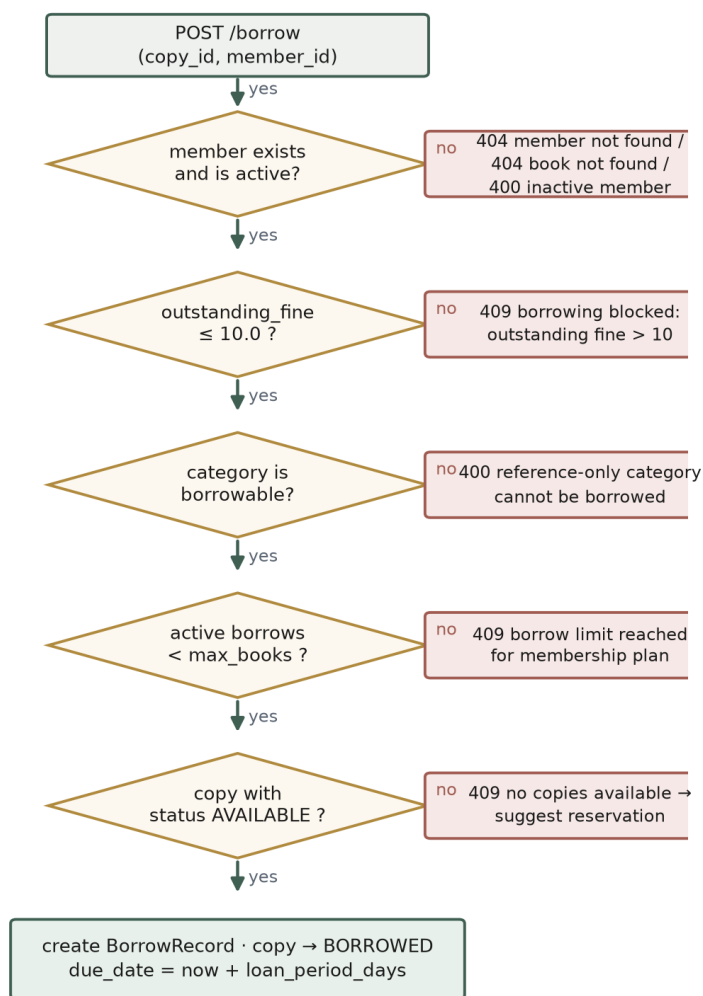
تصمیم‌گیری سرویس امانت‌دهی هنگام ثبت امانت، زنجیره‌ای از پرسش‌های ناب است که در فلوچارت شکل ۳-۴ ترسیم شده است: عضو باید موجود و فعال باشد، جریمه‌ی معوق او از آستانه‌ی ۱۰ دلار فراتر نرود، کتاب مرجع نباشد، ظرفیت عضویت پُر نشده باشد و دست‌کم یک نسخه‌ی موجود وجود داشته باشد. ردِ هر یک از این شرط‌ها با پیام خطای مشخصی به کاربر بازمی‌گردد؛ همین پیام‌های مشخص‌اند که در فصل چهارم مبنای طراحی آزمون‌های منفی قرار می‌گیرند. گزیده‌ی بازنویسی‌شده‌ی این منطق در برنامه ۳-۲ نشان داده شده است.

برنامه ۳-۲: منطق ثبت امانت در متد `borrow_book` (بازنویسی فشرده)

```
def borrow_book(self, member_id: int, copy_id: int) -> BorrowRecord:
    member = self.members.get(member_id)          # 404 if unknown
    copy = self.copies.get(copy_id)               # 404 if unknown
    if not member.is_active:
        raise BusinessRuleError("Member account is inactive")
    if member.outstanding_fine > FINE_BLOCK_THRESHOLD: # threshold = 10.0
        raise BusinessRuleError("Outstanding fines exceed the allowed limit")
    if copy.book.category.is_reference_only:
        raise BusinessRuleError("Reference books cannot be borrowed")
    if self.members.active_borrow_count(member) >= member.type.max_books:
        raise BusinessRuleError("Borrow limit reached for this membership plan")
    try:
        record = BorrowRecord(
            copy_id=copy.id, member_id=member.id,
            borrow_date=utc_now(),
            due_date=utc_now() + timedelta(days=member.type.loan_period_days),
        )
        self.borrows.add(record)
        self.session.flush() # enforces uq_active_borrow_per_copy
    except IntegrityError:    # concurrent borrow of the last copy
        self.session.rollback()
        raise BusinessRuleError("No copies available; you can reserve the book")
    copy.status = CopyStatus.BORROWED
    return record
```

منبع: کد منبع پروژه (app/services/borrowing_service.py)

Decision Flow of BorrowingService.borrow_book



شکل ۳-۴: فلوچارت تصمیم‌گیری سرویس امانت‌دهی در ثبت امانت

محاسبه‌ی جریمه‌ی دیرکرد نیز از قواعد قابل توجه سامانه است: دیرکرد بر پایه‌ی دوره‌های کامل ۲۴ ساعته (و نه روز تقویمی) شمرده می‌شود و مبلغ جریمه از ضرب تعداد دوره‌های دیرکرد در نرخ روزانه‌ی نوع عضویت به دست می‌آید؛ بازگشت در همان روز سررسید جریمه‌ای ندارد. در صورت گم‌شدن نسخه، جریمه‌ای معادل یک‌ونیم برابر قیمت کتاب به حساب عضو افزوده می‌شود و امانت مربوط بسته می‌گردد. جدول ۳-۴ این قواعد را همراه با پارامترهایشان جمع‌بندی کرده است. افزون بر این‌ها، قواعد محافظتی دیگری نیز اعمال می‌شوند: ممنوعیت حذف نویسنده‌ی دارای کتاب، ممنوعیت حذف نسخه‌ی در وضعیت

امانت، یکتایی شایک و منع تغییر برنامه‌ی عضویت به برنامه‌ای که ظرفیتش کمتر از تعداد کتاب‌های در اختیار عضو است.

جدول ۳-۴: قواعد کلیدی کسب‌وکار سامانه و پارامترهای آن‌ها

پارامتر	شرح	قاعده
۱۰ دلار	جریمه‌ی معوق بیش از آستانه، ثبت امانت تازه را مسدود می‌کند	آستانه‌ی توقف امانت
per plan	دوره‌های کامل ۲۴ ساعته‌ی دیرکرد $\times$ نرخ روزانه‌ی نوع عضویت	محاسبه‌ی جریمه‌ی دیرکرد
—	بازگشت در روز سررسید هیچ جریمه‌ای ندارد	بازگشت در سررسید
$price \times 1.5$	یک‌ونیم برابر قیمت کتاب به جریمه‌ی معوق افزوده می‌شود	جریمه‌ی گم‌شدن نسخه
۲ روز	پس از ارتقای رزرو به «آماده»، مهلت برداشت دو روزه است	مهلت برداشت رزرو
شاخص جزئی	هر نسخه حداکثر یک امانت فعال؛ تضمین در سطح پایگاه داده	یکتایی امانت فعال
—	وجود هر رزرو در انتظار برای کتاب، تمدید امانت‌های آن را می‌بندد	انسداد تمدید با صف رزرو
$active \leq new$ max	کاهش سطح عضویت تنها با تعداد کتاب در اختیار مجاز	تغییر برنامه‌ی عضویت

منبع: کلاس BorrowingService و سرویس‌های MemberService و ReservationService

۵-۳ رابط برنامه‌نویسی کاربردی و کنترل دسترسی

سرویس‌دهنده قابلیت‌ها را در هفت گروه مسیریاب ارائه می‌کند: احراز هویت (ثبت‌نام، ورود، پروفایل جاری)، فهرست‌های مرجع (نویسنده، ناشر، موضوع، نوع عضویت)، کتاب‌ها و نسخه‌ها (ایجاد، جست‌وجوی متنی، افزودن و حذف نسخه)، اعضا (ایجاد، مشاهده، پرداخت جریمه، تغییر برنامه‌ی عضویت و خلاصه‌ی پروفایل شخصی)، امانت (ثبت، بازگشت، تمدید، اعلام گم‌شدن)، رزرو (ثبت و لغو) و نتایج آزمون مخصوص مدیر. جست‌وجوی کتاب‌ها به صورت متن آزاد بر سه ستون عنوان، نام نویسنده و موضوع با تطبیق جزئی بدون حساسیت به حروف اجرا می‌شود.

کنترل دسترسی بر پایه‌ی الگوی کنترل دسترسی مبتنی بر نقش [۱۶] پیاده شده است: سه نقش «مدیر»، «کتابدار» و «عضو» در سطح توکن حمل می‌شوند و کارخانه‌ی وابستگی `require_roles` نقش‌های مجاز هر مسیر را اعلام می‌کند. افزون بر کنترل نقشی، قواعد مالکیت در سطح اشیاء نیز بررسی می‌شود؛ برای نمونه، عضو تنها می‌تواند امانت خودش را بازگرداند و تنها برنامه‌ی عضویت خودش را تغییر دهد. جدول ۳-۵ خلاصه‌ی نقش‌ها و دسترسی‌ها را نشان می‌دهد.

جدول ۳-۵: نقش‌های کاربری و سطح دسترسی در سامانه

نقش	شرح	نمونه‌ی دسترسی‌ها
admin	مدیر سامانه	همه‌ی عملیات مدیریت فهرست‌ها، اعضا، پرداخت جریمه، مشاهده‌ی داشبورد نتایج آزمون و اجرای آزمون
librarian	کتابدار	مدیریت امانت و بازگشت، رزروها، فهرست کتاب‌ها، نویسندگان و اعضا؛ بدون داشبورد آزمون
member	عضو کتابخانه	جست‌وجو، مشاهده‌ی جزئیات کتاب، امانت، رزرو، بازگشت امانت خود، تمدید و تغییر برنامه‌ی عضویت خود

منبع: کارخانه‌ی وابستگی `require_roles` در `app/api/deps.py`

۳-۶ امنیت احراز هویت

احراز هویت بر استاندارد توکن وب [۱۵] JSON استوار است. گذرواژه‌ها با الگوریتم `bcrypt` درهم‌سازی می‌شوند و هرگز به‌صورت بازیابی‌پذیر ذخیره نمی‌گردند. پس از ورود، توکن دسترسی با الگوریتم `HS۲۵۶` امضا می‌شود و بار آن شامل شناسه‌ی کاربر، نقش و زمان انقضای ۶۰ دقیقه‌ای است^۶. هر درخواست محافظت‌شده، توکن را از سرآیند `Authorization` به شکل `Bearer` دریافت و اعتبارسنجی می‌کند؛ توکن منقضی، دست‌کاری‌شده یا امضا شده با کلید نادرست با پاسخ ۴۰۱ رد می‌شود و کاربر غیرفعال نیز

^۶ در محیط عملیاتی باید کلید امضای توکن در متغیر محیطی امن نگهداری شود؛ مقدار قرارگرفته در پیکربندی کنونی، مقدار پیش‌فرض محیط توسعه است و پیش از استقرار عمومی باید جایگزین گردد.

— هرچند توکن معتبر داشته باشد — از دسترسی محروم است. امنیت این سازوکار در فصل چهارم با نه آزمون اختصاصی حمله به توکن راستی‌آزمایی می‌شود و اصول دفاع عمقی مطابق توصیه‌های OWASP رعایت گردیده است [۲۰].

۷-۳ رابط کاربری

پیشانه‌ی کاربری هفت نما دارد: ورود، خانه (فهرست کتاب‌ها با جست‌وجو، پالایش موضوع و دسترس‌پذیری)، جزئیات کتاب (وضعیت نسخه‌ها، دکمه‌های امانت و رزرو)، پروفایل عضو (آمار امانت و جریمه، تغییر برنامه‌ی عضویت، برگه‌های امانت‌ها و رزروها)، پروفایل نویسنده، پنل مدیریت (برگه‌های کتاب‌ها، نویسندگان و اعضا همراه با نمودارهای شاخص و فرم‌های افزودن) و صفحه‌ی نتایج آزمون ویژه‌ی مدیر. همه‌ی وضعیت برنامه در یک فروشگاه واحد Zustand گرد آمده است و لایه‌ی سرویس پیشانه، همه‌ی درخواست‌ها را با پیوست خودکار توکن، مدیریت یکدست خطاها و تخت‌سازی پیام‌های اعتبارسنجی سرویس‌دهنده انجام می‌دهد.

نکته‌ی مهندسی قابل توجه در رابط کاربری، سازگاری با محیط آزمون است: انیمیشن‌های ورود مؤلفه‌ها (کتابخانه‌ی framer-motion) در مرورگر بی‌سر می‌توانستند محتوا را در وضعیت شفافیت صفر قفل کنند و آزمون‌های Selenium را شکست دهند؛ به همین دلیل با احترام به تنظیم prefers-reduced-motion، انیمیشن‌ها به‌طور کامل غیرفعال می‌شوند. این تصمیم، هم تجربه‌ی کاربران حساس به حرکت را بهبود می‌دهد و هم قطعیت آزمون‌های خودکار را تضمین می‌کند.

۸-۳ داده‌های اولیه

برای آزمون دستی و اجرای آزمون‌های پایانه‌به‌پایه، اسکریپتی مولد داده‌ی اولیه تهیه شده است که به‌صورت بازگشت‌پذیر (idempotent) سه نوع عضویت، دو ناشر، سه موضوع، ۲۶ نویسنده‌ی واقعی، ۳۰ کتاب با ۹۸ نسخه، یک مدیر، یک کتابدار و ۱۸ عضو با نام‌های فارسی تولید می‌کند. یکی از نسخه‌های

کتاب Clean Code از پیش نزد یک عضو در وضعیت امانت قرار می‌گیرد تا سناریوهای دستی بازگشت و رزرو بدون مقدمه‌چینی قابل اجرا باشند و عضوی نیز با جریمه‌ی معوق سهدلاری ساخته می‌شود تا قاعده‌ی آستانه‌ی جریمه به‌صورت ملموس قابل مشاهده باشد. بازگشت‌پذیری این اسکرپیت بدین معناست که اجرای مکرر آن داده‌ی تکراری نمی‌سازد و همیشه سامانه را به حالت شناخته‌شده و آزمون‌پذیر بازمی‌گرداند — نیازی که هر چارچوب آزمون پایانه‌به‌پایه به آن وابسته است.

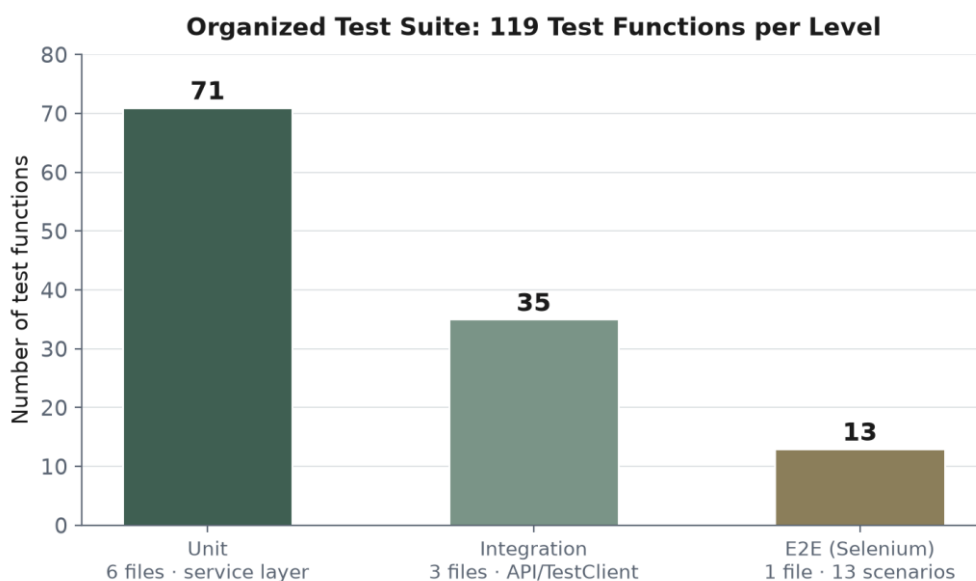
۹-۳ جمع‌بندی فصل

در این فصل، معماری لایه‌ای سامانه، پشته‌ی فناوری، مدل یازده‌جدولی داده با شاخص یکتای جزئی ضدرقابت، ماشین حالت نسخه‌ی کتاب، قواعد کمی امانت‌دهی، کنترل دسترسی مبتنی بر نقش، امنیت توکن، رابط کاربری و داده‌های اولیه شرح داده شد. نقش کلیدی این فصل در روایت کل پایان‌نامه، آشکارسازی «نقاط تماس آزمون» است: هر قاعده و هر خطر فنی معرفی‌شده در اینجا، در فصل چهارم با آزمونی مشخص پوشش داده می‌شود.

فصل چهارم: طراحی و پیاده‌سازی چارچوب آزمون خودکار

۴-۱ استراتژی و ساختار چارچوب

استراتژی آزمون پروژه، ترجمان عملی هرم آزمون است: ۷۱ آزمون واحد در قاعده، ۳۵ آزمون یکپارچگی در میانه و ۱۳ آزمون پایانه‌به‌پایه در رأس؛ شکل ۴-۱ این توزیع را نمایش می‌دهد. آزمون‌ها در سه پوشه‌ی مجزا سازمان یافته‌اند و افزونه‌ی خودکار در فایل `conftest`، بر اساس مسیر هر فایل، نشانگرهای `pytest` متناظر (`unit`، `integration`، `e2e`) را به آزمون‌ها می‌نشانند؛ به این ترتیب اجرای گزینشی هر سطح با یک عبارت مارکر ممکن می‌شود [۱۳]. در پیکربندی `pytest.ini` نیز آستانه‌ی پوشش خط ۸۵ درصد به‌صورت اجباری (`cov-fail-under`) تنظیم شده است تا کاهش پوشش به‌طور خودکار شکست ساخت را در پی داشته باشد.



شکل ۴-۱: توزیع ۱۱۹ آزمون سازمان‌یافته‌ی چارچوب بر اساس سطح

جدول ۴-۱: توزیع آزمون‌ها به تفکیک سطح و فایل

سطح	فایل آزمون	تعداد	تمرکز آزمون
واحد	test_borrowing_service.py	۲۸	قواعد امانت، بازگشت، تمدید و اعلام گم‌شدن
واحد	test_member_service.py	۱۲	تغییر برنامه‌ی عضویت، پرداخت جریمه، ایجاد عضو
واحد	test_book_service.py	۱۱	ایجاد کتاب، افزودن و حذف نسخه
واحد	test_reservation_service.py	۸	ثبت و لغو رزرو و قواعد آن
واحد	test_property_borrowing.py	۹	ویژگی‌های ریاضی جریمه و سررسید (Hypothesis)
واحد	test_factories.py	۳	سلامت کارخانه‌های داده
یکپارچگی	test_api.py	۲۴	جریان‌های API، نقش‌ها و قواعد عضویت
یکپارچگی	test_auth_security.py	۹	حملات و جریمه‌های امنیتی توکن
یکپارچگی	test_concurrency.py	۲	رقابت همزمانی بر آخرین نسخه
پایانه‌به‌پایه	test_e2e.py	۱۳	سناریوهای واقعی کاربر در مرورگر

منبع: پوشه‌ی tests چارچوب آزمون

جزئیات توزیع آزمون‌ها به تفکیک فایل و تمرکز هر فایل در جدول ۴-۱ آمده است. افزون بر این، پنجاه‌وسه آزمون قدیمی‌تر در پوشه‌ی ریشه‌ای tests نگهداری می‌شود که پیش از سازمان‌دهی نهایی نوشته شده‌اند و همچنان به‌عنوان میراث تاریخی اجرا و گزارش می‌گردند.

۴-۲ آزمون‌های واحد

آزمون‌های واحد، سرویس‌های منطق کسب‌وکار را بدون پایگاه داده و بدون چارچوب وب می‌آزمایند. وابستگی‌های هر سرویس با اشیاء ماک از کتابخانه‌ی استاندارد unittest.mock جایگزین می‌شود و کارخانه‌های داده‌ی ساخته‌شده با factory-boy و Faker. ورودی‌های واقع‌نما و غیرتکراری تولید می‌کنند؛ نکته‌ی طراحی مهم آن است که کارخانه‌ها به‌جای مدل‌های ORM، واژه‌نامه‌های ساده تولید

می‌کنند تا آزمون‌های واحد به شکل مدل‌های پایگاه داده وابسته نمانند. برای نمونه، در سطح امانت‌دهی بیست‌وهشت آزمون، همه‌ی مسیرهای فلوچارت شکل ۳-۴ را — از ثبت موفق تا هر پنج نوع خطا، مرز جریمه‌ی ۱۰ دلار در هر دو سمت، و تعیین سررسید درست — پوشش می‌دهند.

نه آزمون ویژگی‌محور با کتابخانه‌ی [۱۹] Hypothesis رفتارهای ریاضی جریمه را به‌صورت ویژگی عمومی بیان می‌کنند: جریمه همواره برابر «روزهای دیرکرد ضربدر نرخ» است؛ بازگشت به‌موقع هرگز جریمه ندارد؛ جریمه هرگز منفی نیست؛ و دیرکرد بیشتر هرگز جریمه‌ی کمتری تولید نمی‌کند. هر ویژگی با پنجاه نمونه‌ی تصادفی از تاریخ‌های بازگشت (۱۴۲۰ تا ۲۰۳۹ میلادی) و نرخ‌های جریمه‌ی پیوسته (۰ تا ۱۰ دلار) آزمون می‌شود و زمان سیستم با freezegun قفل می‌گردد. این رویکرد، مرزهایی چون بازگشت دقیقاً در سررسید و دیرکرد یک‌روزه را با قطعیت بسیار بیشتری از آزمون‌های نمونه‌محور تثبیت می‌کند. نمونه‌ای از این آزمون‌ها در برنامه ۴-۱ آمده است.

برنامه ۴-۱: آزمون ویژگی‌محور محاسبه‌ی جریمه با Hypothesis

```
@given(
    days_late=integers(min_value=1, max_value=365),
    rate=floats(min_value=0.0, max_value=10.0),
)
@settings(max_examples=50, deadline=None)
def test_fine_is_exactly_days_late_times_rate(self, days_late, rate):
    service = make_service(daily_fine_rate=rate)
    borrow = make_mock_borrow_record(due_days_ago=days_late)

    service.return_book(borrow.id)

    assert borrow.fine_amount == pytest.approx(days_late * rate)
```

منبع: کد منبع پروژه (tests/unit/test_property_borrowing.py)

۳-۴ آزمون‌های یکپارچگی

آزمون‌های یکپارچگی، مسیر کامل درخواست را از مسیریاب FastAPI تا پایگاه داده می‌پیمایند، اما دو تمایز مهندسی‌شده با محیط واقعی دارند که قطعیت آزمون را تضمین می‌کنند: نخست، پایگاه داده‌ی SQLite درون‌حافظه‌ای با اتصال ایستا به‌کار گرفته می‌شود و پیش‌نیاز قیده‌های ارجاعی با فعال‌سازی

PRAGMA foreign_keys برقرار می‌گردد؛ دوم، هر آزمون درون یک تراکنش بیرونی اجرا و در پایان پس‌گردانده می‌شود، به‌گونه‌ای که هر آزمون از پایگاه داده‌ی تازه‌ای برخوردار است و نتیجه مستقل از ترتیب اجرای آزمون‌هاست. ساختار این آماده‌ساز در برنامه ۴-۲ نشان داده شده است.

برنامه ۴-۲: آماده‌ساز ایزوله‌سازی پایگاه داده در آزمون‌های یکپارچگی

```
@pytest.fixture(autouse=True)
def db_session(self, db_tables):
    connection = engine.connect()
    transaction = connection.begin()           # single outer transaction
    session = Session(bind=connection)
    app.dependency_overrides[get_db] = lambda: session
    yield session
    session.close()
    transaction.rollback()                     # undo everything the test did
    connection.close()
```

منبع: کد منبع پروژه (tests/integration/conftest.py)

بیست‌وچهار آزمون جریان‌های اصلی API را پوشش می‌دهند: ثبت‌نام و ورود، رمز نادرست، دسترسی بی‌توکن با ۴۰۱، ثبت‌نام تکراری با ۴۰۰، ایجاد و فهرست کتاب‌ها، منع عضو از عملیات مدیریتی، حذف نویسنده‌ی دارای کتاب، چرخه‌ی کامل امانت و بازگشت، اجرای سقف امانت و مجموعه‌ی کامل قواعد تغییر برنامه‌ی عضویت. در کنار آن‌ها، نه آزمون امنیتی توکن، رفتار سامانه را در برابر حملات واقعی راستی‌آزمایی می‌کنند: توکن منقضی، امضای دست‌کاری‌شده، بار نقش‌ارتقا شده بدون امضای مجدد، امضا با کلید نادرست، الگوریتم none، توکن بدون شناسه‌ی کاربر، توکن کاربر ناموجود، توکن معتبر کاربر غیرفعال و رسیدن توکن منقضی به مسیرهای محافظت‌شده — همگی باید با ۴۰۱ رد شوند.

دو آزمون همزمانی، حساس‌ترین خطر داده‌ای سامانه را می‌آزمایند: درخواست هم‌زمان دو عضو برای آخرین نسخه‌ی یک کتاب. این آزمون‌ها با رشته‌های واقعی سیستم‌عامل، نشست‌های جداگانه و پایگاه داده‌ی فایل‌مشترک اجرا می‌شوند و مسابقه برای پایداری نتیجه پنج بار تکرار می‌گردد. نتیجه‌ی درست «فقط یک عضو موفق می‌شود» به لطف شاخص یکتای جزئی معرفی‌شده در بخش ۳-۳ حاصل می‌شود: تلاش دوم در سطح پایگاه داده با خطای یکتایی روبه‌رو می‌شود و سرویس، این خطا را به پیام کسب‌وکاری

«نسخه‌ای موجود نیست؛ می‌توانید رزرو کنید» ترجمه می‌کند. این مورد، نمونه‌ی روشنی از آزمون‌پذیری طراحی است که در فصل سوم آگاهانه تعبیه شد.

۴-۴ آزمون‌های پایانه‌به‌پایه

سیزده آزمون پایانه‌به‌پایه، جریان‌های واقعی کاربر را در مرورگر Chrome بی‌سر و در برابر سامانه‌ی کامل در حال اجرا بازسازی می‌کنند: ورود موفق مدیر و عضو، خطای ورود نادرست، جست‌وجوی کتاب، مشاهده‌ی جزئیات، امانت گرفتن و دیدن کتاب در پروفایل، رزرو کتاب بدون نسخه‌ی آزاد، ایجاد کتاب از پنل مدیریت و مشاهده‌ی آن در فهرست، تغییر برنامه‌ی عضویت و رفت‌و برگشت، و کنترل ناوبری وابسته به نقش (نمایان بودن پیوند مدیریت تنها برای مدیر). اجرای این سناریوها با کتابخانه‌ی [۱۴] Selenium انجام می‌شود.

برای نگه‌داشت‌داری آزمون‌های رابط کاربری، الگوی Page Object به کار رفته است: هر صفحه‌ی برنامه یک کلاس متناظر دارد که مکان‌یاب‌ها و عملیات آن صفحه را در بر می‌گیرد و آزمون‌ها تنها با زبان دامنه (مثلاً login یا open_book_by_title) با صفحه گفت‌وگو می‌کنند؛ تغییر طراحی صفحه تنها کلاس Page Object را متأثر می‌سازد، نه آزمون‌ها را. کلاس پایه نیز انتظارهای صریح (explicit wait)، هم‌نشینی پس از انیمیشن، نگارش متن با سازوکار جایگزین مقداردهی بومی DOM و تهیه‌ی تصویر لحظه‌ای زمان‌دار را فراهم می‌کند.

پایدارسازی آزمون مرورگر بی‌سر، بخشی پرهزینه اما آموزنده از این پروژه بوده است. سه درس اصلی حاصل شد که همگی در پیکربندی نهایی لحاظ شده‌اند: نخست، احترام به prefers-reduced-motion در خود برنامه برای جلوگیری از قفل‌شدن محتوا در شفافیت صفر؛ دوم، اجرای مرورگر در حالت ناشناس تا حباب پنهان «ذخیره‌ی گذرواژه» ورودی‌ها را نبلعد؛ و سوم، قفل کردن زوج نسخه‌ی Chrome

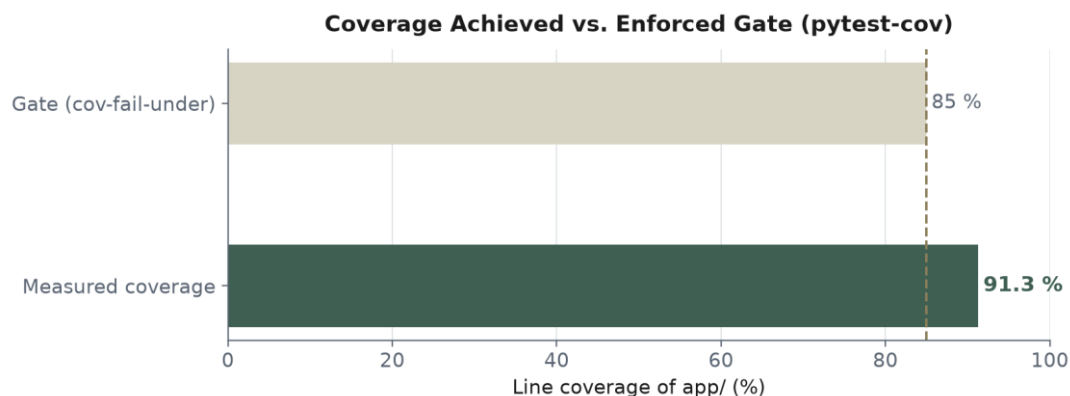
و ^۷ChromeDriver — هر دو با نسخه‌ی ۱۵۲.۰.۷۹۷۷.۷۵ — تا سوءعملکرد بی‌صدای ورودی‌ها رخ ندهد. یافتن راهبرد تعیین درایور نیز در چهار گام انجام می‌شود: مسیر صریح از متغیر محیطی، درایور همراه پروژه، درایور روی PATH و در نهایت نهانگاه Selenium؛ در غیاب همه و بدون اجازه‌ی صریح، اجرا با پیامی راهنما متوقف می‌شود تا دانلود خودکار ناخواسته در شبکه‌های محدود رخ ندهد.

۵-۴ گزارش‌گیری و پوشش‌کد

چارچوب در هر اجرا سه نوع خروجی قابل ردیابی تولید می‌کند: گزارش HTML تفصیلی هر آزمون با `pytest-html`، گزارش JSON ساخت‌یافته با `pytest-json-report` برای مصرف برنامه‌ای، و گزارش پوشش‌کد در سه قالب متنی، JSON و HTML با `pytest-cov`. مجموعه‌ی کامل گزارش‌ها در خط لوله‌ی CI به‌عنوان خروجی اجرا بایگانی می‌شود تا تاریخچه‌ی کیفیت قابل مرور باشد.

پوشش‌خط سرویس‌دهنده در اجرای کامل به حدود ۹۱ درصد رسیده است که از آستانه‌ی اجباری ۸۵ درصد بالاتر است (شکل ۴-۲). افزون بر این، یک داشبورد نتایج آزمون در خود سامانه تعبیه شده است: کاربر مدیر می‌تواند آخرین نتایج `pytest` (تعداد موفق، شکست، ردشده، مدت اجرا و پوشش) را ببیند و با یک کلیک اجرای تازه‌ی آزمون‌های پشتیبان را از رابط کاربری درخواست کند؛ سرویس‌دهنده این درخواست را به‌صورت فرایند جداگانه اجرا و نتیجه را از فایل گزارش می‌خواند. پیوند دادن «سامانه‌ی آزمون‌شونده» به «سامانه‌ی آزمون‌کننده» در یک رابط واحد، گزارش‌گیری را از فایل خام به تصمیم‌پذیری روزمره نزدیک می‌کند.

^۷ نسخه‌ی مرورگر Chrome و درایور `ChromeDriver` باید دقیقاً یکسان باشند؛ ناهماهنگی این زوج در مرورگر بی‌سر (`Headless`) به سقوط بی‌صدای رویدادهای صفحه‌کلید و کلیک منجر می‌شود و آزمون‌ها را ناپایدار می‌کند.



شکل ۴-۲: پوشش خط به دست آمده در برابر آستانه‌ی اجباری اجرا

۴-۶ مقایسه‌ی بازبینی دستی و آزمون خودکار

برای سنجش ارزش چارچوب، سناریوهای دستی متداول پیش از پروژه با آزمون‌های خودکار نگاشت و مقایسه شد. مطابق جدول ۴-۲، گذر کامل دستی از دوازده سناریوی پرتکرار حدود ۱۱ دقیقه به طول می‌انجامد، در حالی که گذر خودکار همان پوشش و بیش از آن را در حدود ۳۰ ثانیه — یعنی نزدیک به ۹۵ درصد سریع‌تر — انجام می‌دهد. با احتساب هزینه‌ی یک‌باره‌ی نگارش آزمون‌ها، نقطه‌ی سربه‌سر پس از حدود سه سیکل بازبینی حاصل می‌شود و از آن پس هر گذر عملاً رایگان است؛ در دهمین سیکل رگرسیون، صرفه‌جویی زمانی از مرز ۹۹ درصد گذر می‌کند.

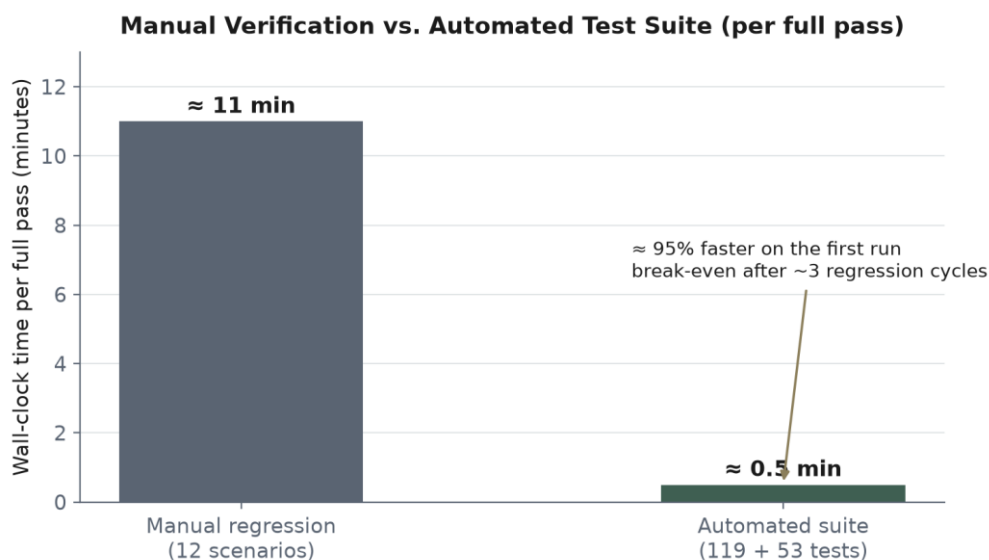
جدول ۴-۲: مقایسه‌ی بازبینی دستی و مجموعه‌ی آزمون خودکار

معیار	بازبینی دستی	آزمون خودکار
زمان هر گذر کامل	حدود ۱۱ دقیقه	حدود ۳۰ ثانیه
دامنه‌ی پوشش	۱۲ سناریوی پرتکرار	۱۱۹ آزمون سازمان‌یافته (+۵۳ میراثی)
تکرارپذیری	وابسته به دقت اپراتور	قطعی و مستقل از اپراتور
زمان کشف رگرسیون	پس از استقرار یا در بازبینی بعدی	در خط لوله، پیش از ادغام تغییر
هزینه‌ی گذرهای بعدی	ثابت و صعودی با رشد محصول	نزدیک به صفر؛ سربه‌سر پس از ۳ [~] سیکل

معیار	بازبینی دستی	آزمون خودکار
سهم پیشنهادی تلاش	۲۰٪ سناریوهای اکتشافی و ارزیابی	۸۰٪ (۴۰ واحد، ۳۰ یکپارچگی، ۱۰ پایانه‌به‌پایه)

منبع: سند مقایسه‌ی manual-vs-automated پروژه (tests/COMPARISON.md)

شکل ۳-۴ این مقایسه‌ی زمانی را تصویر کرده است. نکته‌ی مهم، عدم حذف کامل بازبینی دستی است: بخش اکتشافی و ارزیابی تجربه‌ی کاربر همچنان به داوری انسانی نیاز دارد و سهم پیشنهادی آن حدود ۲۰ درصد تلاش آزمون اعلام شده است.



شکل ۳-۴: مقایسه‌ی زمان هر گذر کامل بازبینی دستی و مجموعه‌ی آزمون خودکار

۴-۷ خط لوله‌ی ادغام مداوم

اجرای آزمون‌ها در گیت‌هاب اکشنز خودکارسازی شده است. خط لوله در هر **push** و هر درخواست ادغام، و همچنین به‌صورت شبانه در ساعت ۰۳:۰۰ UTC اجرا می‌شود و دو شغل موازی دارد: شغل نخست آزمون‌های واحد و یکپارچگی را با آستانه‌ی پوشش اجرا و گزارش‌ها را بایگانی می‌کند؛ شغل دوم — آزمون

پایانه به پایه — سامانه را از پایه برمی سازد و در برابر آن مرورگر می گرداند. جدول ۳-۴ گام های این خط لوله را شرح می دهد.

جدول ۳-۴: گام های خط لوله ی ادغام مداوم پروژه

مرحله	شرح	یادداشت فنی
راه اندازی محیط	دریافت کد و نصب Python ۳.۱۲ با نهانگاه pip	کلید نهانگاه: requirements.txt
آزمون پشتیبان	اجرای pytest به جز آزمون های مرورگر	آستانه ی پوشش ۸۵٪ از pytest.ini
بایگانی گزارش	انتشار گزارش های HTML، JSON و پوشش کد	با شرط always، حتی هنگام شکست
آماده سازی E2E	نصب ۲۰ Node، وابستگی ها و زوج Chrome/ChromeDriver قفل شده	نصب با @puppeteer/browsers؛ نسخه ی ۱۵۲۰.۷۹۷۷.۷۵
اجرای سامانه	تولید داده ی اولیه و برپایی سرویس دهنده و پیشنهاد	انتظار فعال با پایش docs/ و درگاه ۳۰۰۰
اجرای آزمون مرورگر	اجرای ۱۳ سناریوی Selenium	حذف آستانه ی پوشش (سنجش فرایند uvicorn ممکن نیست)
بایگانی شواهد	گزارش e2e، تصاویر لحظه ای و لاگ سرورها	با شرط always برای عیب یابی شکست ها

منبع: فایل github/workflows/ci.yml. پروژه

دو نکته ی عملیاتی از تجربه ی راه اندازی خط لوله شایان ذکر است. نخست آنکه نصب وابستگی های پیشنهادی با npm install (و نه npm ci) انجام می شود^۸. دوم آنکه نخستین اجرای شبانه روی ماشین بار ابری کندتر از محیط محلی، چهار آزمون را به سبب درنگ ناکافی شکست داد؛ این شکست ها با افزایش درنگ های انتظار و به کار بستن سازوکارهای پایداری بخش ۴-۴ رفع شد — نمونه ای واقعی از ارزش اجرای شبانه در کشف وابستگی آزمون ها به محیط اجرا.

^۸ فایل قفل وابستگی های تولید شده در ویندوز شامل وابستگی های اختیاری پلتفرم wasm۳۲-wasi است که نصب قطعی (npm ci) در لینوکس را با خطا مواجه می کند؛ از این رو در خط لوله از npm install استفاده شده است.

۸-۴ جمع‌بندی فصل

در این فصل، چارچوب آزمون خودکار در سه سطح شرح داده شد: آزمون‌های واحد با ماک و کارخانه‌های داده و آزمون ویژگی‌محور، آزمون‌های یکپارچگی با پایگاه داده‌ی ایزوله، مجموعه‌ی حمله‌ی توکن و آزمون رقابت همزمانی، و آزمون‌های پایانه‌به‌پایه با الگوی Page Object و سازوکارهای پایداری مرورگر بی‌سر. گزارش‌گیری چندقابلی، داشبورد نتایج درون سامانه‌ای، پوشش ۹۱ درصدی در برابر آستانه‌ی ۸۵ درصدی و خط لوله‌ی CI دوشغلی، چرخه‌ی کامل «اجرای آزمون، سنجش پوشش و بازخورد سریع» را می‌بندد.

فصل پنجم: نتایج، نتیجه‌گیری و پیشنهادها

۵-۱ مرور دستاوردها

در این پروژه دو خروجی به‌هم‌پیوسته تحقق یافت. در مؤلفه‌ی نخست، سامانه‌ی مدیریت کتابخانه‌ی Aldenwood Library با پیشانه‌ی Next.js و سرویس‌دهنده‌ی FastAPI پیاده شد؛ سامانه‌ای که پوشش کاربردی آن شامل مدیریت کتاب‌ها و نسخه‌ها، اعضا و برنامه‌های عضویت، امانت، بازگشت، تمدید، رزرو، جریمه و کنترل دسترسی سه‌نقشی است. در مؤلفه‌ی دوم، چارچوب آزمون خودکار سه‌سطحی با ۱۱۹ آزمون سازمان‌یافته و ۵۳ آزمون میراثی، پوشش خط حدود ۹۱ درصدی، سه قالب گزارش، داشبورد نتایج درون‌سامانه‌ای و خط لوله‌ی CI شبانه ساخته شد.

دستاوردهای کمی پروژه را می‌توان چنین برشمرد:

۱. پیاده‌سازی ۱۱۹ آزمون سازمان‌یافته در سه سطح (۷۱ واحد، ۳۵ یکپارچگی، ۱۳ پایانه‌به‌پایه) با اجرای کامل موفق؛
۲. پوشش خط حدود ۹۱ درصد سرویس‌دهنده در برابر آستانه‌ی اجباری ۸۵ درصد؛
۳. کاهش زمان گذر کامل بازیابی از حدود ۱۱ دقیقه به حدود ۳۰ ثانیه (حدود ۹۵ درصد سریع‌تر) و سربه‌سر شدن پس از حدود سه سیکل؛
۴. پوشش صریح ریسک‌های حساس: نه سناریوی حمله به توکن، دو آزمون رقابت همزمانی تکرارپذیر و آزمون مرزهای جریمه با روش ویژگی‌محور؛
۵. خودکارسازی کامل اجرا در خط لوله‌ی گیت‌هاب اکشنز با بایگانی گزارش‌ها و اجرای شبانه‌ی آزمون‌های مرورگر.

۲-۵ پاسخ به اهداف پژوهش

اهداف شش‌گانه‌ی اعلام‌شده در بخش ۱-۳ همگی محقق شده‌اند: معماری لایه‌ای تست‌پذیر، استراتژی آزمون بر مبنای هرم، پوشش قواعد کسب‌وکار و رقابت همزمانی، راستی‌آزمایی امنیت توکن، خودکارسازی رابط کاربری با الگوی Page Object، آستانه‌ی اجباری پوشش و خط لوله‌ی ادغام مداوم؛ هر یک با شواهد فصل چهارم قابل راستی‌آزمایی است. پرسش اصلی پژوهش — طراحی چارچوبی چندسطحی، پوشش‌سنج و یکپارچه با CI — از مسیر همان سه‌گانه پاسخ گرفت: «معماری برای آزمون‌پذیری، توزیع آزمون بر مبنای هرم و خودکارسازی اجرا و گزارش».

۳-۵ محدودیت‌ها و پیشنهادهای توسعه

پروژه‌ی حاضر را می‌توان زیربنایی برای گسترش‌های آتی دانست؛ مشروط بر آنکه محدودیت‌های شناخته‌شده‌ی آن صادقانه دیده شود. جدول ۵-۱ مهم‌ترین این محدودیت‌ها را در کنار پیشنهاد مشخص برای رفع هر یک فهرست کرده است.

جدول ۵-۱: محدودیت‌های کنونی و پیشنهادهای توسعه‌ی آتی

محدودیت کنونی	پیشنهاد توسعه
استفاده از SQLite و ذخیره‌سازی توکن در حافظه‌ی مرورگر	مهاجرت به PostgreSQL در CI و جایگزینی توکن با کوکی امن httpOnly و سازوکار بازنشانی توکن
عدم پوشش کارایی و بار سامانه	افزودن آزمون بار با ابزارهایی چون k6 یا Locust و تعریف بودجه‌ی کارایی
عدم پوشش دسترس‌پذیری رابط کاربری	ادغام ممیزی خودکار axe در آزمون‌های پایانه‌به‌پایه
عدم پایش خودکار مسائل امنیتی وابستگی‌ها	افزودن پویش OWASP ZAP در بازه‌های زمانی منظم و تحلیل وابستگی‌ها در CI
ناپایداری ذاتی آزمون‌های Selenium در ماشین‌بارهای کند	بررسی مهاجرت تدریجی به Playwright با انتظارهای خودکار و ردیابی شبکه

پیشنهاد توسعه	محدودیت کنونی
افزودن مقایسه‌ی تصویری نسخه‌ها برای رابط کاربری	نبود آزمون رگرسیون تصویری
اجرای دوره‌ای آزمون جهش و سنجش نرخ کشف عیب مصنوعی	ارزیابی نشده‌ی کیفیت آزمون‌ها فراتر از پوشش خط

منبع: یافته‌های نگارنده در طول پروژه و سند محدودیت‌های مستندات آزمون

۴-۵ نتیجه‌گیری

این پایان‌نامه نشان داد که کیفیت آزمون‌پذیری یک سامانه، نه پیامد تصادفی، بلکه محصول تصمیم‌های آگاهانه‌ی معماری است؛ چنان‌که تفکیک لایه‌ها و تزریق وابستگی در سامانه‌ی موردی، امکان آزمون کامل منطق کسب‌وکار را بدون چارچوب وب و پایگاه داده فراهم آورد. همچنین آشکار شد که هرم آزمون، در مقیاس یک پروژه‌ی کارشناسی نیز راهنمای عملیاتی مؤثری است: توزیع آگاهانه‌ی ۷۱، ۳۵ و ۱۳ آزمون در سه سطح، بازخورد سریع برای توسعه‌دهنده و اطمینان واقع‌نما برای تیم را هم‌زمان ممکن ساخت. خودکارسازی کامل اجرا در خط لوله‌ی ادغام مداوم نیز ارزش خود را در نخستین اجرای شبانه نشان داد، آنجا که وابستگی پنهان آزمون‌ها به سرعت محیط اجرا آشکار و رفع شد.

در مجموع، مسیر طی‌شده — از صورت‌بندی مسئله تا خط لوله‌ی سبز — چارچوبی تکرارپذیر برای نگه‌داشت کیفیت سامانه‌های وب ارائه کرد که هزینه‌ی نگاه‌داشت آن با هر گذر رگرسیون کاهش می‌یابد و با رشد محصول، به‌جای فرسایش، تقویت می‌شود. امید است این کار، الگویی کاربردی برای پروژه‌های مشابه در مقطع کارشناسی باشد و گسترش‌های پیشنهادی بخش ۵-۳، مسیر پژوهش و توسعه‌ی آتی را هموار کند.

مراجع

- [۱] I. Sommerville, Software Engineering, ۱۰th ed. Harlow, U.K.: Pearson, ۲۰۱۶.
- [۲] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, ۸th ed. New York, NY: McGraw-Hill, ۲۰۱۵.
- [۳] G. J. Myers, C. Sandler and T. Badgett, The Art of Software Testing, ۳rd ed. Hoboken, NJ: John Wiley & Sons, ۲۰۱۱.
- [۴] ISTQB, Certified Tester Foundation Level Syllabus, Version ۴.۰. Brussels: International Software Testing Qualifications Board, ۲۰۲۳.
- [۵] ISO/IEC/IEEE ۲۹۱۱۹-۱:۲۰۲۲, Software and Systems Engineering — Software Testing — Part ۱: General Concepts. Geneva: ISO, ۲۰۲۲.
- [۶] M. Fowler, "TestPyramid," martinowler.com, May ۲۰۱۲. [Online]. Available: <https://martinowler.com/bliki/TestPyramid.html>
- [۷] K. Beck, Test-Driven Development: By Example. Boston, MA: Addison-Wesley, ۲۰۰۳.
- [۸] M. Fowler, "Continuous Integration," martinowler.com, Sep. ۲۰۰۶. [Online]. Available: <https://martinowler.com/articles/continuousIntegration.html>
- [۹] K. Beck et al., "Manifesto for Agile Software Development," ۲۰۰۱. [Online]. Available: <https://agilemanifesto.org>
- [۱۰] S. Ramírez, FastAPI Documentation. [Online]. Available: <https://fastapi.tiangolo.com>
- [۱۱] Vercel, Next.js Documentation. [Online]. Available: <https://nextjs.org/docs>
- [۱۲] SQLAlchemy Authors, SQLAlchemy ۲.۰ Documentation. [Online]. Available: <https://docs.sqlalchemy.org>
- [۱۳] H. Krekel, B. Oliveira, R. Pfannschmidt, F. Bruynooghe and others, pytest: Helps You Write Better Programs. [Online]. Available: <https://docs.pytest.org>
- [۱۴] SeleniumHQ, Selenium Browser Automation Documentation. [Online]. Available: <https://www.selenium.dev/documentation>
- [۱۵] M. Jones, J. Bradley and N. Sakimura, JSON Web Token (JWT), IETF RFC ۷۵۱۹, May ۲۰۱۵.
- [۱۶] R. S. Sandhu, E. J. Coyne, H. L. Feinstein and C. E. Youman, "Role-Based Access Control Models," IEEE Computer, vol. ۲۹, no. ۲, pp. ۳۸-۴۷, Feb. ۱۹۹۶.
- [۱۷] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA: Addison-Wesley, ۲۰۰۲.
- [۱۸] A. Hunt and D. Thomas, The Pragmatic Programmer: Your Journey to Mastery, ۲۰th Anniversary ed. Boston, MA: Addison-Wesley, ۲۰۱۹.
- [۱۹] D. R. MacIver et al., Hypothesis: Property-Based Testing for Python. [Online]. Available: <https://hypothesis.readthedocs.io>
- [۲۰] OWASP Foundation, OWASP Top ۱۰:۲۰۲۱. [Online]. Available: <https://owasp.org/Top۱۰>

پیوست الف: راهنمای نصب و اجرا

گام‌های لازم برای راه‌اندازی محلی سامانه و اجرای آزمون‌ها در جدول الف-۱ آمده است. پیش‌نیازها عبارت‌اند از Node.js، Python ۳.۱۲، نسخه‌ی ۲۰ به بالا و مرورگر Chrome. مقادیر پیش‌فرض ورود نیز در صفحه‌ی ورود سامانه نمایش داده می‌شود: admin/Admin۱۲۳! برای مدیر و member۱/Member۱۲۳! برای عضو نمونه.

جدول الف-۱: گام‌های نصب و اجرای پروژه

گام	فرمان	شرح
۱	git clone <repo-url> && cd LibraryTestFramework	دریافت مخزن پروژه و ورود به پوشه‌ی چارچوب آزمون
۲	pip install -r requirements.txt	نصب وابستگی‌های سرویس‌دهنده و چارچوب آزمون
۳	python seed_db.py	تولید داده‌های اولیه‌ی بازگشت‌پذیر
۴	uvicorn app.main:app --reload	اجرای سرویس‌دهنده روی درگاه ۸۰۰۰
۵	cd Library && npm install && npm run dev	نصب و اجرای پیشانه روی درگاه ۳۰۰۰
۶	pytest -m "not e2e and not selenium"	اجرای آزمون‌های واحد و یکپارچگی همراه با سنجش پوشش
۷	pytest tests/e2e -m e2e (ChromeDriver)	اجرای آزمون‌های مرورگر (نیازمند سرورهای فعال و ChromeDriver)
۸	pytest --html=tests/reports/report.html --self-contained-html	تولید گزارش HTML یکپارچه

منبع: مستندات README چارچوب آزمون

پیوست ب: واژه‌نامه

جدول ب-۱ معادل‌های فارسی به کاررفته در این پایان‌نامه را در کنار اصطلاح انگلیسی و توضیح کوتاه

هر یک جمع‌بندی می‌کند.

جدول ب-۱: واژه‌نامه‌ی اصطلاحات فنی پایان‌نامه

اصطلاح انگلیسی	معادل فارسی	توضیح
Unit Test	آزمون واحد	آزمون کوچک‌ترین مؤلفه به صورت مجزا از وابستگی‌ها
Integration Test	آزمون یکپارچگی	آزمون تعامل اجزای به هم پیوسته‌ی سامانه
End-to-End (E2E) Test	آزمون پایانه به پایانه	آزمون رفتار کل سامانه از دید کاربر نهایی
Test Pyramid	هرم آزمون	راهنمای توزیع آزمون‌ها: فراوانی واحد، کمیابی پایانه به پایانه
Regression Testing	آزمون رگرسیون	اجرای مجدد آزمون‌ها پس از تغییر برای کشف خرابی‌های نو
Mock	شیء ماک	شیء جایگزین برای شبیه‌سازی وابستگی در آزمون
Fixture	آماده‌ساز آزمون	بستر داده و محیط ثابت موردنیاز اجرای آزمون
Code Coverage	پوشش کد	نسبت خطوط اجراشده‌ی کد در حین آزمون
Page Object Model	الگوی شیء صفحه	جداسازی مکان‌یاب‌ها و عملیات صفحه از منطق آزمون
Headless Browser	مرورگر بی سر	مرورگر بدون رابط گرافیکی، مناسب اجرای خودکار
Continuous Integration	ادغام مداوم	ادغام و آزمون خودکار مکرر تغییرات در شاخه‌ی اصلی
JWT	توکن وب JSON	استاندارد توکن امضا شده برای احراز هویت
RBAC	کنترل دسترسی مبتنی بر نقش	اعطای مجوز بر اساس نقش کاربر
ORM	نگاشت شیء-رابطه‌ای	پل میان اشیاء برنامه و جدول‌های رابطه‌ای
Partial Unique Index	شاخص یکتای جزئی	قید یکتایی صرفاً بر ردیف‌های واجد شرط
Property-Based Testing	آزمون ویژگی‌محور	آزمون رفتار عمومی با نمونه‌های تصادفی فراوان
Idempotent	بازگشت پذیر	خاصیت اجرای مکرر بدون اثر انباشتی

اصطلاح انگلیسی	معادل فارسی	توضیح
Seed Data	داده‌های اولیه	داده‌های شناخته‌شده‌ی آغازین برای اجرای آزمون‌پذیر سامانه

منبع: نگارنده، با الهام از واژگان سیلابس [۴] ISTQB